Actividad 8

Algoritmos Voraces (Greedy)

Mochila fraccional y Cobertura de antenas

Estudiante	Rodrigo
Institución	Universidad La Salle - Arequipa
Curso	Análisis de Algoritmos
Lenguaje	Python 3
Contenido	Explicación de la estrategia, análisis de complejidad, ejemplos de ejecución y validación

1. Introducción

Un **algoritmo voraz (greedy)** construye la solución paso a paso. En cada paso toma la decisión que parece mejor en ese momento (la *elección local óptima*) y nunca la revisa. No funciona para cualquier problema: solo es correcto cuando el problema cumple dos propiedades:

- **Propiedad de elección voraz:** existe una solución óptima que contiene la elección local que hace el algoritmo.
- **Subestructura óptima:** después de hacer esa elección, lo que queda es un problema más pequeño del mismo tipo, cuya solución óptima se combina con la elección hecha.

En esta actividad se resuelven dos problemas que cumplen ambas propiedades. Para cada uno se explica la estrategia, se justifica por qué es óptima, se analiza su complejidad con notación Big O y se muestran ejecuciones reales del código incluido en el repositorio.

Estructura del repositorio

```
main.py                    Programa principal (menú interactivo)
src/mochila_fraccional.py  Problema 1
src/cobertura_antenas.py   Problema 2
tests/test_greedy.py       17 pruebas de validación
docs/                      Informe PDF y salidas de ejecución
README.md                  Cómo ejecutar el proyecto
```

2. Problema 1: Mochila fraccional

2.1 Enunciado resumido

Se tiene una mochila con capacidad máxima **W** y **n** objetos, cada uno con un valor v_i y un peso w_i. Cada objeto existe una sola vez, pero se puede tomar una **fracción** de él. Se debe maximizar el valor total sin superar W.

2.2 Estrategia Greedy utilizada

Criterio: la **densidad de valor** $d_i = v_i / w_i$ (cuánto valor aporta cada unidad de peso). Los objetos se toman de mayor a menor densidad:

- Si el objeto cabe completo en el espacio restante, se toma completo.
- Si no cabe, se toma solo la fracción que llena exactamente la capacidad restante y el algoritmo termina, porque la mochila queda llena.

Pseudocódigo

```
MochilaFraccional(objetos, W):
    ordenar objetos por (valor / peso) de mayor a menor
    restante <- W ; valorTotal <- 0
    para cada objeto o en el orden obtenido:
        si restante = 0: terminar
        tomado <- min(o.peso, restante)
        valorTotal <- valorTotal + o.valor * (tomado / o.peso)
        restante <- restante - tomado
    retornar valorTotal
```

2.3 ¿Por qué es óptimo? (argumento de intercambio)

Supongamos una solución óptima que deja de usar peso de un objeto A de mayor densidad mientras usa peso de un objeto B de menor densidad. Se puede mover una pequeña cantidad ε de peso de B a A. El valor cambia en $\varepsilon \cdot (d_A - d_B) \geq 0$, es decir, no empeora. Repitiendo este intercambio se llega a la solución que construye el algoritmo voraz, así que esta también es óptima. Este argumento **depende de poder tomar fracciones**.

Observación: en la mochila 0/1 (sin fracciones) la misma estrategia falla. Con W = 50 y los objetos A(60, 10), B(100, 20), C(120, 30), el voraz toma A y B y obtiene 160, pero lo óptimo es B + C = 220. Por eso la versión 0/1 se resuelve con programación dinámica.

2.4 Código (función principal)

```
def mochila_fraccional(objetos, capacidad):
    """
    Resuelve la mochila fraccional con estrategia Greedy.

    Parametros:
        objetos   -- lista de Objeto
        capacidad -- capacidad maxima de la mochila (>= 0)

    Retorna:
        (seleccion, valor_total) donde seleccion es una lista de tuplas
        (objeto, peso_tomado) en el orden en que fueron elegidos.
    """
    capacidad = Fraction(str(capacidad))
    if capacidad < 0:
        raise ValueError("La capacidad no puede ser negativa.")

    # PASO 1: ordenar por densidad descendente -> O(n log n).
    # sorted() es estable: ante empates se respeta el orden de entrada.
    ordenados = sorted(objetos, key=lambda o: o.densidad, reverse=True)

    seleccion = []
    valor_total = Fraction(0)
    restante = capacidad

    # PASO 2: recorrer los objetos en orden de densidad -> O(n).
    for obj in ordenados:
        if restante == 0:
            break  # mochila llena, no hay nada mas que hacer

        if obj.peso <= restante:
            # El objeto cabe completo: tomarlo entero.
            tomado = obj.peso
        else:
            # Solo cabe una parte: tomar la fraccion que llena la mochila.
            tomado = restante

        valor_total += obj.valor * tomado / obj.peso
        restante -= tomado
        seleccion.append((obj, tomado))

    return seleccion, valor_total
```

2.5 Traza paso a paso del Ejemplo 1 (capacidad 50)

Objeto	Valor	Peso	Valor/Peso
A	60	10	6
B	100	20	5
C	120	30	4

Paso	Objeto	Capacidad restante antes	Se toma	Valor acumulado
1	A	50	10/10 (completo)	60
2	B	40	20/20 (completo)	160
3	C	20	20/30 (fracción 2/3)	160 + 120·(20/30) = 240

Con A y B la mochila queda con 20 unidades libres; C pesa 30, así que solo se toma 20/30 de C. La mochila queda llena y el valor total es 240.

2.6 Análisis de complejidad

Parte del algoritmo	Costo	Justificación
Cálculo de la densidad de cada objeto	O(n)	Una división por objeto.
Ordenamiento por densidad	O(n log n)	Ordenamiento por comparación (sorted de Python usa Timsort).
Recorrido para llenar la mochila	O(n)	Un solo ciclo; cada objeto se procesa una vez con operaciones O(1).
Total (tiempo)	**O(n log n)**	O(n) + O(n log n) + O(n) = O(n log n): el ordenamiento domina.
Espacio	**O(n)**	Lista ordenada y lista de selección.

Si los objetos ya vinieran ordenados por densidad, el algoritmo costaría solo O(n). Con un montículo (heap) también podría hacerse en O(n + k log n) si solo se extraen los k objetos necesarios, pero el caso general sigue siendo O(n log n).

2.7 Ejemplos de ejecución

Salida real obtenida con `python main.py --ejemplos`:

Ejemplo guía 1: capacidad 50; A(60,10), B(100,20), C(120,30)

```
Objetos seleccionados:
  A completo
  B completo
  20/30 Parte del objeto C
Valor total obtenido: 240
```

Ejemplo guía 2: capacidad 25; A(80,20), B(100,10), C(120,30)

```
Objetos seleccionados:
  B completo
  15/20 Parte del objeto A
Valor total obtenido: 160
```

Las densidades son B = 10, A = 4 y C = 4. A y C empatan, pero el resultado (160) es el mismo con cualquier orden entre ellos, porque tienen la misma densidad.

Caso propio: capacidad 15 con 7 objetos A(10,2), B(5,3), C(15,5), D(7,7), E(6,1), F(18,4), G(3,1)

```
Objetos seleccionados:
  E completo
  A completo
  F completo
  C completo
  G completo
  2/3 Parte del objeto B
Valor total obtenido: 55.33
```

El orden por densidad es E (6), A (5), F (4.5), C (3), G (3), B (1.67), D (1). D nunca se usa. B entra a 2/3 y aporta 5·(2/3) = 3.33. Total: 6 + 10 + 18 + 15 + 3 + 3.33 = 55.33.

Casos borde:

```
MOCHILA - Caso borde: capacidad 0
Objetos seleccionados:
  (ninguno)
Valor total obtenido: 0

MOCHILA - Caso borde: capacidad mayor al peso total (capacidad 100)
Objetos seleccionados:
  B completo
  A completo
Valor total obtenido: 30
```

3. Problema 2: Cobertura de antenas

3.1 Enunciado resumido

Hay **n** casas en posiciones enteras sobre una línea recta. Una antena colocada en x cubre el intervalo **[x − R, x + R]**. Se debe colocar la **menor cantidad** de antenas de modo que todas las casas queden cubiertas.

3.2 Estrategia Greedy utilizada

Criterio: procesar las casas de izquierda a derecha y, para la primera casa aún sin cubrir, poner la antena **lo más a la derecha posible** sin dejar esa casa fuera: en **x = casa + R**. Esa antena cubre desde la casa hasta *casa + 2R*. Se saltan las casas cubiertas y se repite.

La idea es que la casa más a la izquierda sin cubrir *tiene* que ser cubierta por alguna antena. Entre todas las posiciones que la cubren, la de casa + R es la que llega más lejos hacia la derecha, así que cubre al menos tantas casas como cualquier otra opción.

Pseudocódigo

```
CoberturaAntenas(casas, R):
    ordenar casas y quitar repetidas
    antenas <- lista vacía ; i <- 0
    mientras i < n:
        x <- casas[i] + R            // antena lo más a la derecha posible
        agregar x a antenas
        mientras i < n y casas[i] <= x + R:
            i <- i + 1               // saltar casas ya cubiertas
    retornar antenas
```

3.3 ¿Por qué es óptimo? (el voraz "va por delante")

Sean $g_1 < g_2 < \ldots < g_k$ las antenas del voraz y $o_1 < \ldots < o_m$ las de cualquier solución válida. Se demuestra por inducción que **el alcance derecho de las primeras j antenas del voraz es mayor o igual que el de las primeras j antenas de cualquier otra solución**:

- **Base:** la primera casa p_1 debe ser cubierta por alguna antena $o_1 \leq p_1 + R$, por lo que su alcance es a lo sumo $p_1 + 2R$, que es justo el de g_1.
- **Paso:** la primera casa que queda fuera del alcance de las j antenas del voraz también queda fuera del de la otra solución, que necesita otra antena para ella; esa antena no llega más lejos que la elegida por el voraz.

Entonces el voraz nunca necesita más antenas que cualquier otra solución, es decir, $k \leq m$, y por lo tanto es óptimo. Además, las pruebas incluyen una comparación contra fuerza bruta (sección 4).

3.4 Código (función principal)

```
def cobertura_antenas(casas, R):
    """
    Calcula la menor cantidad de antenas para cubrir todas las casas.

    Parametros:
        casas -- lista de posiciones enteras de las casas
        R     -- rango de cobertura de cada antena (R >= 0)

    Retorna:
        Lista con las posiciones de las antenas colocadas.
    """
    if R < 0:
        raise ValueError("El rango de cobertura R no puede ser negativo.")

    # PASO 1: ordenar y eliminar duplicados (dos casas en la misma posicion
    # se cubren juntas) -> O(n log n).
    posiciones = sorted(set(casas))

    antenas = []
    i = 0
    n = len(posiciones)

    # PASO 2: un solo recorrido; cada casa se visita una vez -> O(n).
    while i < n:
        primera_sin_cubrir = posiciones[i]

        # Eleccion Greedy: antena lo mas a la derecha posible.
        x = primera_sin_cubrir + R
        antenas.append(x)

        # Esta antena cubre [x - R, x + R]. Saltar todas las casas cubiertas.
        limite = x + R
        while i < n and posiciones[i] <= limite:
            i += 1

    return antenas
```

3.5 Traza paso a paso del Ejemplo 1 (R = 5)

Casas ordenadas: [1, 2, 7, 11, 20, 21, 30]

Paso	Primera casa sin cubrir	Antena en (casa + R)	Cobertura [x − R, x + R]	Casas cubiertas
1	1	6	[1, 11]	1, 2, 7, 11
2	20	25	[20, 30]	20, 21, 30

Resultado: antenas en 6 y 25, cantidad total = 2.

3.6 Análisis de complejidad

Parte del algoritmo	Costo	Justificación
Eliminar repetidas (set)	O(n)	Cada casa se inserta una vez en la tabla hash.
Ordenamiento de las posiciones	O(n log n)	Ordenamiento por comparación.
Ciclo principal (while + while interno)	O(n)	El índice i solo avanza y nunca retrocede. Cada casa se visita exactamente una vez, aunque haya dos ciclos anidados (análisis amortizado). Colocar una antena es O(1).
Total (tiempo)	**O(n log n)**	O(n) + O(n log n) + O(n): el ordenamiento domina.
Espacio	**O(n)**	Lista ordenada de posiciones y lista de antenas.

Aunque el ciclo interno está dentro del externo, no es $O(n^2)$: el número total de incrementos de *i* entre ambos ciclos es n. Si la entrada ya estuviera ordenada, el costo bajaría a O(n).

3.7 Ejemplos de ejecución

Ejemplo guía 1: casas [1,2,7,11,20,21,30], R = 5

```
Antenas colocadas aproximadamente en:
  6
  25
Cantidad total: 2
```

Ejemplo guía 2: casas [2,4,8,15,18,22], R = 3

```
Antenas colocadas aproximadamente en:
  5
  18
  25
Cantidad total: 3
```

Traza: 2 → antena en 5 (cubre [2, 8]: casas 2, 4, 8); 15 → antena en 18 (cubre [15, 21]: casas 15, 18); 22 → antena en 25 (cubre [22, 28]). Total: 3.

Caso propio: casas desordenadas y con repetidas [30,1,2,2,7,11,21,20,50,52], R = 5

```
Antenas colocadas aproximadamente en:
  6
  25
  55
Cantidad total: 3
```

Al ordenar y quitar repetidas queda [1, 2, 7, 11, 20, 21, 30, 50, 52]. Las dos primeras antenas son las mismas del ejemplo guía 1 y las casas 50 y 52 necesitan una tercera antena, en 55.

Casos borde:

```
ANTENAS - Caso borde: R = 0 (una antena por casa distinta)
Casas: [4,4,9,12]
Antenas colocadas aproximadamente en:
  4
  9
  12
Cantidad total: 3

ANTENAS - Caso borde: una sola casa, R = 3
Antenas colocadas aproximadamente en:
  13
Cantidad total: 1

ANTENAS - Caso borde: todas las casas en un solo intervalo de largo 2R
Casas: [10,12,14,16,18,20], R = 5
Antenas colocadas aproximadamente en:
  15
Cantidad total: 1
```

4. Validación de casos

Las pruebas están en `tests/test_greedy.py` y se ejecutan con `python -m unittest discover -v`. Resultado: **17 pruebas, todas aprobadas**.

Problema	Caso validado	Qué comprueba
Mochila	Ejemplos 1 y 2 de la guía	Valor 240 y 160 y el orden de selección.
Mochila	Capacidad 0	No se selecciona nada; valor 0.
Mochila	Capacidad mayor que el peso total	Se toman todos los objetos completos.
Mochila	Lista de objetos vacía	Retorna selección vacía y valor 0.
Mochila	Pesos y capacidad decimales	Cálculo exacto con fracciones.
Mochila	Peso ≤ 0 o capacidad negativa	Lanza ValueError (entrada inválida).
Mochila	200 casos aleatorios	Nunca se excede la capacidad.
Antenas	Ejemplos 1 y 2 de la guía	Posiciones [6, 25] y [5, 18, 25].
Antenas	Sin casas / una sola casa	Lista vacía / una antena.
Antenas	Casas desordenadas y repetidas	El resultado no depende del orden de entrada.
Antenas	R = 0	Una antena por cada posición distinta.
Antenas	R negativo	Lanza ValueError.
Antenas	300 casos aleatorios vs. fuerza bruta	Todas las casas quedan cubiertas y la cantidad de antenas es igual al mínimo real.

Salida de la ejecución de las pruebas

```
test_R_cero (tests.test_greedy.TestAntenas.test_R_cero) ... ok
test_R_negativo (tests.test_greedy.TestAntenas.test_R_negativo) ... ok
test_desordenadas_y_repetidas (tests.test_greedy.TestAntenas.test_desordenadas_y_repetidas) ... ok
test_ejemplo_1 (tests.test_greedy.TestAntenas.test_ejemplo_1) ... ok
test_ejemplo_2 (tests.test_greedy.TestAntenas.test_ejemplo_2) ... ok
test_optimalidad_vs_fuerza_bruta (tests.test_greedy.TestAntenas.test_optimalidad_vs_fuerza_bruta)
Compara con fuerza bruta en 300 casos aleatorios pequenos. ... ok
test_sin_casas (tests.test_greedy.TestAntenas.test_sin_casas) ... ok
test_una_casa (tests.test_greedy.TestAntenas.test_una_casa) ... ok
test_capacidad_cero (tests.test_greedy.TestMochila.test_capacidad_cero) ... ok
test_capacidad_mayor_que_todo (tests.test_greedy.TestMochila.test_capacidad_mayor_que_todo) ... ok
test_capacidad_negativa (tests.test_greedy.TestMochila.test_capacidad_negativa) ... ok
test_decimales (tests.test_greedy.TestMochila.test_decimales) ... ok
test_ejemplo_1 (tests.test_greedy.TestMochila.test_ejemplo_1) ... ok
test_ejemplo_2 (tests.test_greedy.TestMochila.test_ejemplo_2) ... ok
test_nunca_excede_capacidad (tests.test_greedy.TestMochila.test_nunca_excede_capacidad) ... ok
test_peso_invalido (tests.test_greedy.TestMochila.test_peso_invalido) ... ok
test_sin_objetos (tests.test_greedy.TestMochila.test_sin_objetos) ... ok
----------------------------------------------------------------------
Ran 17 tests in 0.033s
OK
```

5. Cómo ejecutar el proyecto

```
python main.py              # menú interactivo
python main.py --ejemplos   # ejemplos guía de la actividad
python -m unittest discover -v   # pruebas de validación
```

Solo se usa la biblioteca estándar de Python 3.8 o superior. En el menú, la mochila se ingresa como `nombre valor peso` (uno por línea) y las casas como una lista de enteros separados por espacios o comas.

6. Conclusiones

- Ambos problemas cumplen la propiedad de elección voraz, por eso una decisión local (mayor valor/peso; antena lo más a la derecha posible) lleva a la solución global óptima.
- En los dos casos el ordenamiento domina el costo y ambos algoritmos son **O(n log n)** en tiempo y **O(n)** en espacio.
- La mochila fraccional es óptima con greedy solo porque se permiten fracciones; en la versión 0/1 hay que usar programación dinámica.
- Se validaron los resultados con los ejemplos de la guía, casos borde y una comparación contra fuerza bruta.